Python Algorithm Templates for Competitive Programming

Contest template, fast I/O, algorithm templates, complexity tables, and a gotchas checklist. Strings, lists, math, and the standard library are in the companion Methods and Libraries Reference. Python 3.10+ assumed; newer-version features are marked.

1. Template and fast I/O

```
import sys
from collections import deque, defaultdict, Counter
import heapq, bisect, math
from itertools import permutations, combinations, accumulate

input = sys.stdin.readline          # much faster than built-in input()
sys.setrecursionlimit(10**6)        # default ~1000; deep recursion is still risky
INF = float("inf")                  # or 10**18 for an int infinity
MOD = 10**9 + 7

def main():
    n = int(input())
    a = list(map(int, input().split()))
    # solve here
    print(ans)

main()                              # code inside a function runs faster than at
module level
```

Input patterns

```
n = int(input())                            # one int
a, b = map(int, input().split())            # two ints on one line
arr = list(map(int, input().split()))       # a line of ints
s = input().strip()                         # string (strip the trailing \n)
grid = [input().strip() for _ in range(n)]  # n lines
mat = [list(map(int, input().split())) for _ in range(n)]

data = sys.stdin.read().split()             # ALL tokens at once (fastest for big
input)
it = iter(data)
n = int(next(it))

for line in sys.stdin:                      # until EOF
    ...
```

Output patterns

```
print(*arr)                                 # space-separated
print("\n".join(map(str, res)))             # best: build once, print once
sys.stdout.write("\n".join(out) + "\n")     # same, slightly faster
print(f"{x:.6f}")                           # fixed decimals
print("YES" if ok else "NO")
```

Two values per line (a list of pairs), joined with newlines and printed once:

```
res = [(1, 2), (3, 4), (5, 6)]
print("\n".join(f"{a} {b}" for a, b in res))   # 1 2 / 3 4 / 5 6, one pair per line
```

- `sys.stdin.readline` keeps the newline. `int()` and `.split()` ignore it; for strings use `.strip()`.
- Thousands of `print()` calls are slow. Collect results in a list and print once.
- Don't mix `input = sys.stdin.readline` with `sys.stdin.read()` carelessly; pick one style per program.

Faster input, interactive problems, and deep recursion

```
# bytes input: skips text decoding; int() accepts bytes directly
input = sys.stdin.buffer.readline
n = int(input())
a = list(map(int, input().split()))
s = input().decode().strip()            # decode when you need a real str

data = sys.stdin.buffer.read().split()  # fastest for huge input (tokens are bytes)
it = iter(data)
n = int(next(it))

# interactive problems: flush after every output, and never read to EOF
print(ans, flush=True)
sys.stdout.flush()

sys.stderr.write("debug\n")             # debug output that won't mix into the answer

# 3.11+: lift the 4300-digit limit on int <-> str conversion
if hasattr(sys, "set_int_max_str_digits"):
    sys.set_int_max_str_digits(0)

# deep recursion: run main in a thread with a bigger stack
import threading
threading.stack_size(2**27)             # 128 MB; lower it if the judge limits memory
sys.setrecursionlimit(10**6)
threading.Thread(target=main).start()   # use instead of calling main() directly
```

- With `sys.stdin.buffer`, tokens are `bytes`. `int(b"12")` works, but `tok == "abc"` is always `False`, so decode first.
- After `sys.stdin.read()` the input is consumed, so a later `input()` returns nothing. Pick one style per program.
- `sys.exit()` stops immediately, but a bare `except:` swallows it.
- Raising `setrecursionlimit` alone can still crash the interpreter on very deep recursion. The thread trick is the usual fix; an explicit stack is safer still.

2. Algorithm templates

Conventions used below: nodes are numbered 0 to n-1 (convert 1-indexed input with `u -= 1`), `g` is an adjacency list, and `INF`, `deque`, `heapq`, `bisect`, `math`, `accumulate` come from the template in section 1. Typical graph input is `n m` on the first line, then `m` lines of `u v` (or `u v w` for weighted edges).

```
n, m = map(int, input().split())
g = [[] for _ in range(n)]          # unweighted: g[u] = [v, ...]
for _ in range(m):
```

```
    u, v = map(int, input().split())
    u -= 1; v -= 1                  # 1-indexed input -> 0-indexed
    g[u].append(v)
    g[v].append(u)                  # drop this line for a directed graph

gw = [[] for _ in range(n)]         # weighted: gw[u] = [(v, w), ...]
for _ in range(m):
    u, v, w = map(int, input().split())
    gw[u-1].append((v-1, w))
    gw[v-1].append((u-1, w))
```

BFS on a grid

- **Input:** `grid` is a list of strings, one per row (`grid = [input().strip() for _ in range(R)]`). `(sr, sc)` is the 0-indexed start cell. `'#'` marks a wall; change that test to match your problem's blocked symbol.
- **Output:** `dist`, an R x C list of lists: steps from the start to each cell, or `-1` if unreachable.

```
def bfs(grid, sr, sc):
    R, C = len(grid), len(grid[0])
    dist = [[-1] * C for _ in range(R)]
    dist[sr][sc] = 0
    q = deque([(sr, sc)])
    while q:
        r, c = q.popleft()
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] != "#" and dist[nr][nc]
== -1:
                dist[nr][nc] = dist[r][c] + 1
                q.append((nr, nc))
    return dist
```

BFS on a graph

- **Input:** `g`, the unweighted adjacency list built above, and `s`, the 0-indexed start node.
- **Output:** `dist`, a list of length n: fewest edges from `s` to each node, or `-1` if unreachable. Only valid when every edge costs the same.

```
def bfs(g, s):
    dist = [-1] * len(g)
    dist[s] = 0
    q = deque([s])
    while q:
        u = q.popleft()
        for v in g[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist
```

Iterative DFS (avoids the recursion limit)

- **Input:** `g`, the adjacency list, and `s`, the start node.

- **Output:** `order`, a list of the nodes visited, in visiting order. It covers only the component containing `s`. To count components, share one `seen` list and call the search from every unseen node.

```
def dfs(g, s):
    seen = [False] * len(g)
    seen[s] = True
    stack, order = [s], []
    while stack:
        u = stack.pop()
        order.append(u)
        for v in g[u]:
            if not seen[v]:
                seen[v] = True
                stack.append(v)
    return order
```

Dijkstra

- **Input:** `g`, the weighted adjacency list (`gw` above), where `g[u]` is a list of `(v, w)` tuples, and `s`, the start node. Weights must be non-negative.
- **Output:** `dist`, a list of length n with the shortest distance from `s`, or `INF` if unreachable. To recover paths, also set `parent[v] = u` whenever you relax an edge.

```
def dijkstra(g, s):                  # g[u] = [(v, w), ...]
    dist = [INF] * len(g)
    dist[s] = 0
    h = [(0, s)]
    while h:
        d, u = heapq.heappop(h)
        if d > dist[u]:
            continue                 # stale entry
        for v, w in g[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(h, (dist[v], v))
    return dist
```

Topological sort (Kahn)

- **Input:** `g`, a directed adjacency list (only `g[u].append(v)`, no reverse edge), and `n`, the node count.
- **Output:** `order`, a list where every edge goes from an earlier node to a later one. If `len(order) < n`, the graph has a cycle.

```
def topo(g, n):
    indeg = [0] * n
    for u in range(n):
        for v in g[u]:
            indeg[v] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    order = []
    while q:
        u = q.popleft()
```

```
        order.append(u)
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return order                    # len(order) < n means a cycle
```

Union-Find (DSU)

- **Input:** `n`, the number of elements, numbered 0 to n-1. Each starts in its own set.
- **Methods:** `find(x)` returns the representative of x's set. `union(a, b)` merges two sets and returns `True` if they were separate.
- **Typical use:** `d = DSU(n)`, call `d.union(u, v)` for each edge, test connectivity with `d.find(a) == d.find(b)`, and read a component's size with `d.sz[d.find(x)]`.

```
class DSU:
    def __init__(self, n):
        self.p = list(range(n))
        self.sz = [1] * n
    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]]   # path halving
            x = self.p[x]
        return x
    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return False
        if self.sz[a] < self.sz[b]:
            a, b = b, a
        self.p[b] = a
        self.sz[a] += self.sz[b]
        return True
```

Prefix sums

- **Input:** `a`, a list of numbers (1D), or `g`, an n x m list of lists (2D). Query bounds `l`, `r` are 0-indexed and inclusive.
- **Output:** `pre`, a list of length `len(a) + 1` where `pre[i]` is the sum of the first i items; or `P`, an (n+1) x (m+1) table. Each query then takes O(1).

```
pre = [0] + list(accumulate(a))      # pre[i] = sum of a[:i]
range_sum = pre[r + 1] - pre[l]      # sum of a[l..r] inclusive

# 2D
P = [[0] * (m + 1) for _ in range(n + 1)]
for i in range(n):
    for j in range(m):
        P[i+1][j+1] = g[i][j] + P[i][j+1] + P[i+1][j] - P[i][j]
# sum of rows r1..r2, cols c1..c2:
# P[r2+1][c2+1] - P[r1][c2+1] - P[r2+1][c1] + P[r1][c1]
```

Two pointers / sliding window

- **Input:** `a`, a list or string of length n, and a rule for when a window is invalid (for example, sum above K, or a repeated character). You fill in `window_is_bad` and the add/remove steps.
- **Output:** `best`, such as the longest valid window. It works when shrinking from the left can only help, which holds for non-negative numbers or distinct-character rules.

```
l = 0
for r in range(n):
    # add a[r] to the window
    while window_is_bad:
        # remove a[l] from the window
        l += 1
    best = max(best, r - l + 1)
```

Binary search on the answer

- **Input:** `ok(x)`, a function you write that returns `True` or `False` and is monotonic (once `True`, it stays `True` for larger x), and bounds `lo`, `hi` that bracket the answer.
- **Output:** `lo`, the smallest x with `ok(x)` true. For the largest x where ok is true, use `mid = (lo + hi + 1) // 2` with `lo = mid` / `hi = mid - 1`. For floats, loop a fixed 100 times instead.

```
lo, hi = 0, 10**18                  # invariant: answer in [lo, hi]
while lo < hi:
    mid = (lo + hi) // 2
    if ok(mid):                     # ok is monotonic: False...False True...True
        hi = mid
    else:
        lo = mid + 1
# lo is the smallest value where ok is True
```

Sieve of Eratosthenes

- **Input:** `n`, the largest number to test, an int (practical up to about 10^7).
- **Output:** a list of all primes up to n. Keep `is_p` too if you need O(1) checks: `is_p[i]` is `1` when i is prime.

```
def sieve(n):
    is_p = bytearray([1]) * (n + 1)
    is_p[0:2] = b"\x00\x00"
    for i in range(2, math.isqrt(n) + 1):
        if is_p[i]:
            is_p[i*i::i] = bytearray(len(range(i*i, n + 1, i)))
    return [i for i in range(n + 1) if is_p[i]]
```

Primes and divisors

- **Input:** `n`, a positive int.
- **Output:** `factorize` returns a dict `{prime: exponent}`, so `factorize(12)` is `{2: 2, 3: 1}`. `divisors` returns a sorted list of every divisor. Each call is O(sqrt n), fine for n up to about 10^12 when called once.

```
def factorize(n):
    f = {}
```

```
    d = 2
    while d * d <= n:
        while n % d == 0:
            f[d] = f.get(d, 0) + 1
            n //= d
        d += 1
    if n > 1:
        f[n] = f.get(n, 0) + 1
    return f

def divisors(n):
    out = []
    for i in range(1, math.isqrt(n) + 1):
        if n % i == 0:
            out.append(i)
            if i != n // i:
                out.append(n // i)
    return sorted(out)
```

Modular arithmetic and nCr

- **Input:** `MOD`, a prime (usually 10^9 + 7), and `N`, the largest n you will ever need. `nCr(n, r)` takes ints with 0 <= r <= n <= N.
- **Output:** `nCr` returns C(n, r) mod MOD. Build `fact` and `inv` once, then each query is O(1). The Fermat inverse `pow(a, MOD - 2, MOD)` only works when MOD is prime.

```
pow(a, b, MOD)                      # a^b mod MOD
pow(a, MOD - 2, MOD)                # inverse when MOD is prime
pow(a, -1, MOD)                     # inverse in general (3.8+)

N = 10**6 + 1
fact = [1] * N
for i in range(1, N):
    fact[i] = fact[i-1] * i % MOD
inv = [1] * N
inv[-1] = pow(fact[-1], MOD - 2, MOD)
for i in range(N - 1, 0, -1):
    inv[i-1] = inv[i] * i % MOD

def nCr(n, r):
    if r < 0 or r > n:
        return 0
    return fact[n] * inv[r] % MOD * inv[n-r] % MOD
```

Fenwick tree (point update, prefix query)

- **Input:** `n`, the number of positions. Indices are 1-indexed. Build with `bit = BIT(n)`, then `bit.add(i, a[i-1])` for each element.
- **Output:** `add(i, v)` adds v at position i. `query(i)` returns the sum of positions 1 to i, so a range sum is `query(r) - query(l - 1)`. Both are O(log n).

```
class BIT:
    def __init__(self, n):
        self.n = n
```

```
        self.t = [0] * (n + 1)
    def add(self, i, v):                # 1-indexed
        while i <= self.n:
            self.t[i] += v
            i += i & -i
    def query(self, i):                 # sum of 1..i
        s = 0
        while i > 0:
            s += self.t[i]
            i -= i & -i
        return s
```

Longest increasing subsequence, O(n log n)

- **Input:** `a`, a list of numbers.
- **Output:** `len(tails)`, the length of the longest strictly increasing subsequence. `tails[k]` is the smallest possible last value of an increasing subsequence of length k + 1, so `tails` is not the subsequence itself.

```
tails = []
for x in a:
    i = bisect.bisect_left(tails, x)     # bisect_right for non-decreasing
    if i == len(tails):
        tails.append(x)
    else:
        tails[i] = x
len(tails)
```

Common DP shapes

- **Input:** for knapsack, `items`, a list of `(weight, value)` tuples, and `W`, the capacity. For coin change, `coins`, a list of denominations, and `T`, the target amount.
- **Output:** `dp[W]`, the best total value, or `dp[T]`, the fewest coins (`INF` if impossible). Iterate capacity downward when each item can be used once, and upward when items can be reused.

```
# 1D knapsack (0/1): iterate capacity DOWNWARD
dp = [0] * (W + 1)
for w, v in items:
    for c in range(W, w - 1, -1):
        dp[c] = max(dp[c], dp[c - w] + v)

# Unbounded knapsack / coin change (min coins): iterate UPWARD
dp = [INF] * (T + 1); dp[0] = 0
for c in range(1, T + 1):
    for coin in coins:
        if coin <= c:
            dp[c] = min(dp[c], dp[c - coin] + 1)

# Edit distance / LCS: 2D table dp[i][j] over prefixes of two strings
```

Floyd-Warshall (all-pairs shortest paths)

- **Input:** `n` nodes and `dist`, an n x n list of lists with `0` on the diagonal, the edge weight where an edge exists, and `INF` elsewhere. Weights may be negative.

- **Output:** `dist[i][j]` becomes the shortest distance from i to j (`INF` if unreachable). If any `dist[i][i] < 0`, there is a negative cycle. It is O(n^3), so n up to about 400 in Python.

```
dist = [[INF] * n for _ in range(n)]
for i in range(n):
    dist[i][i] = 0
for u, v, w in edges:                    # 0-indexed; keep the cheapest parallel edge
    dist[u][v] = min(dist[u][v], w)

def floyd(dist, n):
    for k in range(n):
        dk = dist[k]
        for i in range(n):
            dik = dist[i][k]
            if dik == INF:
                continue
            di = dist[i]
            for j in range(n):
                if dik + dk[j] < di[j]:
                    di[j] = dik + dk[j]
    return dist
```

Kruskal (minimum spanning tree)

- **Input:** `n` nodes numbered 0 to n-1, and `edges`, a list of `(w, u, v)` tuples with the weight first so sorting works. Uses the `DSU` class above.
- **Output:** `total`, the weight of the spanning tree, and `used`, the chosen edges. If `len(used) < n - 1`, the graph is disconnected and you got a spanning forest.

```
def kruskal(n, edges):                   # edges = [(w, u, v), ...]
    edges.sort()
    d = DSU(n)
    total, used = 0, []
    for w, u, v in edges:
        if d.union(u, v):
            total += w
            used.append((u, v, w))
    return total, used
```

Segment tree (point update, range query)

- **Input:** `a`, a list of values, `op`, any associative function of two values (sum, min, max, gcd, ...), and `e`, its identity (`0` for sum, `INF` for min, `0` for gcd).
- **Methods:** `update(i, v)` sets `a[i] = v`. `query(l, r)` combines `a[l:r]` with `op`, half-open, so `r` is excluded. Both are O(log n); building is O(n).
- **Examples:** `st = SegTree(a)` for sums, `st = SegTree(a, min, INF)` for range minimum, `st = SegTree(a, math.gcd, 0)` for range gcd.

```
class SegTree:
    def __init__(self, a, op=lambda x, y: x + y, e=0):
        self.n = len(a)
        self.op, self.e = op, e
        self.t = [e] * self.n + list(a)
        for i in range(self.n - 1, 0, -1):
```

```
            self.t[i] = op(self.t[2*i], self.t[2*i+1])
    def update(self, i, v):             # set a[i] = v
        i += self.n
        self.t[i] = v
        while i > 1:
            i >>= 1
            self.t[i] = self.op(self.t[2*i], self.t[2*i+1])
    def query(self, l, r):              # op over a[l:r]
        left = right = self.e
        l += self.n
        r += self.n
        while l < r:
            if l & 1:
                left = self.op(left, self.t[l])
                l += 1
            if r & 1:
                r -= 1
                right = self.op(self.t[r], right)
            l >>= 1
            r >>= 1
        return self.op(left, right)
```

Range updates (add to a whole range) need lazy propagation, which is a different template.

LCA with binary lifting

- **Input:** `g`, an undirected adjacency list of a tree (0-indexed, as built above), and `root`. For queries, two nodes `u` and `v`.
- **Output:** `build_lca` returns `depth` (distance from the root) and `up`, where `up[k][v]` is the 2^k-th ancestor of v. `lca(u, v, depth, up)` returns the lowest common ancestor. The distance between two nodes is `depth[u] + depth[v] - 2 * depth[lca]`. Build is O(n log n), each query O(log n).

```
def build_lca(g, root=0):
    n = len(g)
    LOG = max(1, n.bit_length())
    up = [[root] * n for _ in range(LOG)]
    depth = [0] * n
    seen = [False] * n
    seen[root] = True
    stack = [root]
    while stack:
        u = stack.pop()
        for v in g[u]:
            if not seen[v]:
                seen[v] = True
                depth[v] = depth[u] + 1
                up[0][v] = u
                stack.append(v)
    for k in range(1, LOG):
        prev, cur = up[k-1], up[k]
        for v in range(n):
            cur[v] = prev[prev[v]]
    return depth, up
```

```
def lca(u, v, depth, up):
    if depth[u] < depth[v]:
        u, v = v, u
    diff = depth[u] - depth[v]
    k = 0
    while diff:                          # lift u to v's depth
        if diff & 1:
            u = up[k][u]
        diff >>= 1
        k += 1
    if u == v:
        return u
    for k in range(len(up) - 1, -1, -1): # lift both until just below the LCA
        if up[k][u] != up[k][v]:
            u, v = up[k][u], up[k][v]
    return up[0][u]
```

KMP and Z-function (string matching)

- **Input:** `s`, `text`, and `pat` are strings (or lists).
- **Output:** `prefix_function(s)` returns `pi`, where `pi[i]` is the length of the longest proper prefix of `s[:i+1]` that is also its suffix. `find_all(text, pat)` returns every start index where `pat` occurs in `text`. `z_function(s)` returns `z`, where `z[i]` is the length of the longest common prefix of `s` and `s[i:]`. All are O(n).
- **Tip:** the smallest period of `s` is `len(s) - pi[-1]`, and `s` is a repetition of a shorter block when that divides `len(s)`.

```
def prefix_function(s):
    pi = [0] * len(s)
    for i in range(1, len(s)):
        k = pi[i-1]
        while k and s[i] != s[k]:
            k = pi[k-1]
        if s[i] == s[k]:
            k += 1
        pi[i] = k
    return pi

def find_all(text, pat):                 # '#' must not appear in either string
    m = len(pat)
    pi = prefix_function(pat + "#" + text)
    return [i - 2*m for i in range(len(pi)) if pi[i] == m]

def z_function(s):
    n = len(s)
    z = [0] * n
    l = r = 0
    for i in range(1, n):
        if i < r:
            z[i] = min(r - i, z[i - l])
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1
        if i + z[i] > r:
```

```
            l, r = i, i + z[i]
    return z
```

String hashing (substring equality)

- **Input:** `s`, a string. You build the tables once, then compare any two substrings in O(1).
- **Output:** `build_hash(s)` returns `h` and `p` (prefix hashes and powers of the base). `get(h, p, l, r)` returns the hash of `s[l:r]` (half-open). Two substrings are equal, with very high probability, when their hashes match.
- The modulus `2**61 - 1` is large enough that collisions are unlikely, and the random base defeats tests built against a fixed one. Needs `import random`.

```
import random
M = (1 << 61) - 1
B = random.randrange(10**5, 10**9)

def build_hash(s):
    n = len(s)
    h = [0] * (n + 1)
    p = [1] * (n + 1)
    for i, ch in enumerate(s):
        h[i+1] = (h[i] * B + ord(ch)) % M
        p[i+1] = p[i] * B % M
    return h, p

def get(h, p, l, r):                    # hash of s[l:r]
    return (h[r] - h[l] * p[r-l]) % M

# s[a:a+k] == s[b:b+k]  <=>  get(h, p, a, a+k) == get(h, p, b, b+k)
```

3. Complexity and speed

Operation	Cost
`list` append / pop at end	O(1)
`list` insert / pop at front, `x in list`, `remove`	O(n)
`list` slice, copy	O(k)
`list.sort()` / `sorted()`	O(n log n)
`deque` append / popleft	O(1)
`dict` / `set` get, add, `in`	O(1) average
`heapq` push / pop	O(log n)
`bisect` search	O(log n); `insort` is O(n)
`str` concatenation in a loop	O(n^2) total; use `join`

Rough input-size guide (CPython, ~10^7 simple ops per second)

n	Usually need
up to 10	O(n!) brute force
up to 20	O(2^n) bitmask
up to 500	O(n^3)
up to 5,000	O(n^2) (tight in Python)
up to 10^5	O(n log n)
up to 10^6	O(n)
up to 10^9 and beyond	O(log n) or O(1) math

Speed tips

- Put code inside a function. Local variables are much faster than globals.
- Use PyPy if the judge offers it. It is often 5 to 50 times faster on loop-heavy code, but recursion and some library calls can behave differently.
- Prefer built-ins, comprehensions, `join`, `sum`, `sorted` over manual loops.
- Read all input at once and print all output at once.
- Use `array` or `bytearray` for big numeric tables when memory is tight.
- Avoid recursion for depth above about 10^4; convert to an explicit stack.

4. Gotchas checklist

- `sort()` and `reverse()` return `None`. `a = a.sort()` wipes your list.
- `[[0] * m] * n` makes n references to the same row. Use a comprehension.
- `b = a` is the same list, not a copy. Use `a.copy()`, and `copy.deepcopy` for nested lists.
- Removing items while iterating skips elements. Iterate over a copy or build a new list.
- `list.pop(0)` and `insert(0, x)` are O(n). Use `deque`.
- `if s.find("x"):` is a bug. `-1` is truthy and `0` is falsy. Use `in`.
- `strip("abc")` removes any of those characters, not the substring. Use `removeprefix` / `removesuffix`.
- `/` is always float division. Use `//` for integers, and watch precision on big values.
- `round(2.5)` is `2` (banker's rounding). Use `int(x + 0.5)` for round-half-up on non-negatives.
- Never compare floats with `==`. Use `math.isclose`.
- `heapq` is min-only. Negate for a max-heap.
- `bisect` needs a sorted list. `groupby` needs sorted input.
- `@cache` / `lru_cache` need hashable arguments. Pass tuples, not lists.
- Clear caches between test cases if the function depends on global state.
- Mutable default arguments (`def f(x=[])`) are shared across calls.
- Reading with `readline` leaves `\n` on strings. Strip before comparing.
- Sets and dicts can't contain lists. Use tuples.
- `int(input())` fails on trailing spaces only in rare formats; `.strip()` if unsure.

- Off-by-one: slices exclude `stop`, and `range(a, b)` excludes `b`.
- Deep recursion can crash the interpreter even after raising the limit. Prefer iterative versions.

5. Contest-day checklist

1. Read the constraints first and pick the complexity target (section 3).
2. Paste the template (section 1) and confirm the input format.
3. Test the samples, then edge cases: n = 1, empty input, all equal, max values, negatives, duplicates.
4. Check for overflow-like issues only in other languages; in Python watch speed instead.
5. Brute force small cases against your solution if you have time.
6. Before submitting: remove debug prints, and check output format (spaces, newlines, decimals).